

得物前端一面

得物前端一面

1. 自我介绍
2. 长列表优化方案?
3. React的setState是同步还是异步，为什么?
4. fiber介绍一下 为什么能实现空闲时执行低优先级任务?
5. commonjs和esm区别，循环依赖
6. 原型链
7. 原型
8. this指向
9. Call,apply, bind区别
10. 柯里化 应用
11. 闭包 应用
12. 内存泄漏? 什么时候产生内存泄漏说一下知道的hook
13. 写过什么hook，介绍一下
14. 函数组件怎么实现componentDidMount,componentDidUnmountuseEffect原理 为什么不能放在条件表达式里面
15. Type 和interface区别
16. EventLoop
17. 防抖和节流 应用
18. requestAnimationFrame介绍
19. promise写一个最大并发
20. react写一个todo Lis

参考答案：

2. 长列表优化方案

1. 虚拟滚动 (Virtual Scrolling)：只渲染视口内的元素，随着滚动动态加载和卸载元素，常用库有 React Virtualized 和 Vue Virtual Scroll List。
2. 懒加载 (Lazy Loading)：在用户滚动到一定位置时加载更多数据，减少初始加载时间和内存占用。
3. 分页加载：将数据分为多页，用户通过分页控件加载数据，减少单次渲染的压力。
4. 服务器端渲染 (SSR)：初次加载时在服务器端生成 HTML，减轻客户端渲染压力。
5. 批量更新 (Batch Updates)：合并多次 DOM 更新操作，减少重绘重排次数，提高渲染性能。

3. React 的 `setState` 是同步还是异步，为什么？

React 的 `setState` 在合成事件处理和生命周期方法中是异步的，在原生事件和定时器中是同步的。异步更新有助于优化性能，减少不必要的渲染。

4. Fiber 介绍

React Fiber 是 React 的协调引擎，允许 React 在浏览器主线程空闲时执行低优先级任务，通过时间切片技术将渲染工作分割成更小的任务块，从而实现任务的分片执行和优先级调度。

5. CommonJS 和 ESM 区别，循环依赖

- CommonJS：同步导入，适用于服务器端，如 Node.js。支持动态导入，导出是对象的引用。
- ESM (ES Modules)：静态导入，适用于现代浏览器和服务端，导出是值的拷贝。编译时确定模块依赖，支持异步加载。

循环依赖在 CommonJS 中可以部分解决（返回未完成的模块对象），但在 ESM 中会导致模块初始化失败。

6. 原型链

JavaScript 通过原型链实现对象的继承，所有对象都有一个 `proto` 属性指向其构造函数的原型对象，原型对象也可以有自己的 `proto` 属性，从而形成原型链。

7. 原型

原型是一个对象，所有通过构造函数创建的对象都会共享原型上的属性和方法。

8. `this` 指向

- 全局上下文：指向 `window`（浏览器）或 `global`（Node.js）。
- 函数调用：严格模式下 `undefined`，非严格模式下 `window`。
- 方法调用：指向调用该方法的对象。
- 构造函数调用：指向新创建的对象。
- 箭头函数：没有自己的 `this`，继承自外层作用域。

9. call, apply, bind 区别

- call: 改变函数 `this` 指向, 立即执行, 参数逐个传递。
- apply: 改变函数 `this` 指向, 立即执行, 参数以数组形式传递。
- bind: 改变函数 `this` 指向, 不立即执行, 返回一个新的函数。

10. 柯里化 应用

柯里化是将一个接受多个参数的函数转换为接受一个参数的函数, 并返回接受剩余参数的新函数。应用场景包括参数复用、延迟执行和函数组合。

```
1 const curry = (fn) => (...args) =>
2   args.length >= fn.length
3     ? fn(...args)
4     : (...moreArgs) => curry(fn)(...args, ...moreArgs);
5
6 const add = (a, b) => a + b;
7 const curriedAdd = curry(add);
8 console.log(curriedAdd(1)(2)); // 3
```

11. 闭包 应用

闭包是指有权访问另一个函数作用域中变量的函数, 常用于创建私有变量和实现函数工厂。

```
1 function createCounter() {
2   let count = 0;
3   return function() {
4     count++;
5     return count;
6   };
7 }
8
9 const counter = createCounter();
10 console.log(counter()); // 1
11 console.log(counter()); // 2
```

12. 内存泄漏

内存泄漏是指由于疏忽或错误导致程序无法释放已不再使用的内存, 常见的情况包括:

- 未清理的定时器或事件监听器
- 闭包中的变量引用

- 未清理的DOM引用

13. 常见的 Hook

- `useState`: 状态管理。
- `useEffect`: 副作用处理。
- `useRef`: 获取DOM元素或保存变量。
- `useMemo`: 性能优化, 避免不必要的计算。
- `useCallback`: 性能优化, 避免不必要的函数创建。

自定义 Hook 介绍

自定义Hook是将逻辑提取到一个可复用的函数中, 通常用于提取重复逻辑或复杂状态管理。

```
1 function useCounter(initialValue = 0) {
2   const [count, setCount] = useState(initialValue);
3   const increment = () => setCount(count + 1);
4   const decrement = () => setCount(count - 1);
5   return { count, increment, decrement };
6 }
```

14. 函数组件实现 `componentDidMount` 和 `componentDidUnmount`

通过 `useEffect` 实现:

```
1 useEffect(() => {
2   // componentDidMount
3   return () => {
4     // componentDidUnmount
5   };
6 }, []);
```

`useEffect` 原理, 为什么不能放在条件表达式中

`useEffect` 的原理是依赖数组变化时执行回调函数, 不能放在条件表达式中是因为Hooks必须在组件的顶层调用, 保证每次渲染时Hooks的调用顺序一致。

15. Type 和 Interface 区别

- `Type`: 类型别名, 可以用于基本类型、联合类型和元组。
- `Interface`: 接口定义, 可以继承和实现, 适用于对象类型。

16. Event Loop

JavaScript运行时的机制，事件循环不断从任务队列中取出任务执行，保证单线程非阻塞的异步执行模型。

17. 防抖和节流 应用

- 防抖（Debounce）：延迟执行函数，最后一次操作后延迟指定时间执行，常用于搜索输入。
- 节流（Throttle）：规定时间间隔内只执行一次函数，常用于滚动、缩放事件。

```
1 // 防抖
2 function debounce(fn, delay) {
3   let timer;
4   return function(...args) {
5     clearTimeout(timer);
6     timer = setTimeout(() => fn.apply(this, args), delay);
7   };
8 }
9
10 // 节流
11 function throttle(fn, interval) {
12   let lastTime = 0;
13   return function(...args) {
14     const now = Date.now();
15     if (now - lastTime >= interval) {
16       fn.apply(this, args);
17       lastTime = now;
18     }
19   };
20 }
```

18. requestAnimationFrame 介绍

`requestAnimationFrame` 是浏览器提供的一个方法，用于在下一次重绘之前执行指定的回调函数，通常用于实现平滑的动画效果。

19. Promise 实现最大并发

```
1 function promiseWithConcurrency(tasks, concurrency) {
2   let running = 0;
3   let index = 0;
4   const results = [];
5   return new Promise((resolve, reject) => {
6     function runNext() {
```

```

7     while (running < concurrency && index < tasks.length) {
8         const currentIndex = index;
9         const task = tasks[index++];
10        running++;
11        task()
12            .then(result => {
13                results[currentIndex] = result;
14            })
15            .catch(err => {
16                reject(err);
17            })
18            .finally(() => {
19                running--;
20                if (index === tasks.length && running === 0) {
21                    resolve(results);
22                } else {
23                    runNext();
24                }
25            });
26    }
27 }
28 runNext();
29 });
30 }

```

20. React 实现一个 Todo List

```

1  import React, { useState } from 'react';
2
3  function TodoList() {
4      const [todos, setTodos] = useState([]);
5      const [input, setInput] = useState('');
6
7      const addTodo = () => {
8          setTodos([...todos, input]);
9          setInput('');
10     };
11
12     const removeTodo = (index) => {
13         setTodos(todos.filter((_, i) => i !== index));
14     };
15
16     return (
17         <div>
18             <input

```

```
19     type="text"
20     value={input}
21     onChange={(e) => setInput(e.target.value)}
22   />
23   <button onClick={addTodo}>Add Todo</button>
24   <ul>
25     {todos.map((todo, index) => (
26       <li key={index}>
27         {todo} <button onClick={() => removeTodo(index)}>Remove</button>
28       </li>
29     ))}
30   </ul>
31 </div>
32 );
33 }
34
35 export default TodoList
```